Ex. No. 08	DATA PERSISTENCE USING SQLITE AND FIREBASE
Date of Exercise	08.09.2026
Git Hub Host Link	https://github.com/gurubruhh/Flutter-Laboratory.git

Aim

To implement data persistence using a local SQLite database and cloud Firebase database in a Flutter application.

Description

This experiment demonstrates how to store and manage student data in a Flutter application using SQLite and Firebase**.** SQLite is used for storing data locally on the device, while Firebase Firestore is used for storing data in the cloud. The application performs CRUD operations such as creating, reading, updating, and deleting student records. It also displays the stored student details using a list and verifies data persistence.

Part A – SQLite

Procedure:

1. Created a Flutter project named `sqlite_app`.
2. Added `sqflite` and `path` packages.
3. Created `database_helper.dart` for database operations.
4. Created a `students` table with `id`, `name`, and `course` fields.
5. Implemented CRUD operations:
    - Create – Insert student
    - Read – Display students
    - Update – Modify student details
    - Delete – Remove student
6. Collected student details using `TextField`.
7. Displayed stored students using `ListView.builder()`.
8. Verified that the data remains available after restarting the application.

Part B – Firebase

Procedure:

1. Created a Firebase project using Firebase Console.
2. Connected the Flutter application with Firebase.
3. Added the required Firebase packages.
4. Initialized Firebase in the Flutter application.
5. Created a `students` collection in Cloud Firestore.
6. Inserted student information into Firebase.
7. Retrieved Firebase records and displayed them in the Flutter application.

Program

database_helper.dart

```
import 'package:path/path.dart';

import 'package:sqflite_common_ffi/sqflite_ffi.dart';

class DatabaseHelper {

  static final DatabaseHelper instance = DatabaseHelper._init();

  static Database? _database;

  DatabaseHelper._init();

  Future<Database> get database async {

  if (_database != null) return _database!;

    _database = await openDB();

    return _database!;

  }

  Future<Database> openDB() async {

    final dbPath = await getDatabasesPath();

    final path = join(dbPath, 'students.db');
```

```
  return await databaseFactory.openDatabase(
    path,
    options: OpenDatabaseOptions(
      version: 1,
      onCreate: (db, version) async {
        await db.execute('''
        CREATE TABLE students(
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            name TEXT,
            course TEXT
          )
        ''');
      },
    ),
  );
}
Future<int> insertStudent(String name, String course) async {
  final db = await database;
  return db.insert('students', {
    'name': name,
    'course': course,
  });
}
```

```
  Future<List<Map<String, dynamic>>> getStudents() async {
    final db = await database;
    return db.query('students');
  }
  Future<int> updateStudent(
      int id, String name, String course) async {
    final db = await database;
    return db.update(
      'students',
      {'name': name, 'course': course},
      where: 'id = ?',
      whereArgs: [id],
    );
  }
  Future<int> deleteStudent(int id) async {
    final db = await database;
    return db.delete(
      'students',
where: 'id = ?',
      whereArgs: [id],
    );
  }
}
```

main.dart

```
import 'package:flutter/material.dart';
import 'package:sqflite_common_ffi/sqflite_ffi.dart';
import 'database_helper.dart';

void main() {
  sqfliteFfiInit();
  databaseFactory = databaseFactoryFfi;
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
    debugShowCheckedModeBanner: false,
    home: const StudentPage(),
    );
  }
}

class StudentPage extends StatefulWidget {
  const StudentPage({super.key});

  @override
  State<StudentPage> createState() => _StudentPageState();
}

class _StudentPageState extends State<StudentPage> {
```

```
final name = TextEditingController();
final course = TextEditingController();

List<Map<String, dynamic>> students = [];

void loadStudents() async {
  students = await DatabaseHelper.instance.getStudents();
  setState(() {});
}

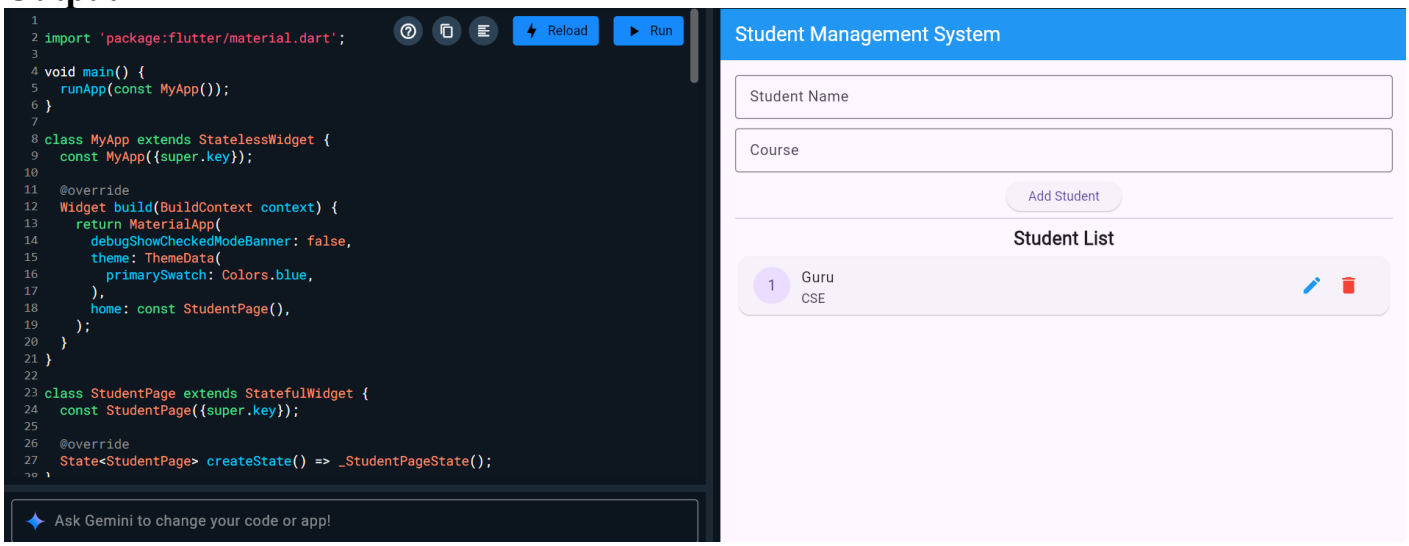
void addStudent() async {
  await DatabaseHelper.instance.insertStudent(
    name.text,
    course.text,
  );

  name.clear();
  course.clear();

  loadStudents();
}
@override
void  initState()  {
  super.initState();
  loadStudents();
}
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Student SQLite App'),
    ),
    body: Padding(
      padding: const EdgeInsets.all(16),
```

```
      child: Column(
        children: [
        TextField(
          controller: name,
          decoration:
              const InputDecoration(labelText: 'Student Name'),
        ),
        TextField(
          controller: course,
          decoration:
              const InputDecoration(labelText: 'Course'),
        ),
        ElevatedButton(
          onPressed: addStudent,
          child: const Text('Add Student'),
        ),
        const SizedBox(height: 20),
        Expanded(
          child: ListView.builder(
            itemCount: students.length,
            itemBuilder: (context, index) {
            return ListTile(
              title: Text(students[index]['name']),
              subtitle: Text(students[index]['course']),
            );
            },
          ),
        ),
      ],
    ),
  ),
 );
 }
}
```

Output



Result

Thus, the Flutter application successfully implemented **data persistence using SQLite**. Student details were inserted into the local database and retrieved using ListView.builder(). The CRUD operations were successfully demonstrated.